



PROJECT PROPOSAL

Mini Shopping Mall

Arcade-Idle Store Tycoon — Game Design and Project Proposal

You are the whole staff of a mall, one unit at a time.

IDLE TYCOON

Prepared by	Kaon — developer and designer
Revision	2 · 23 September 2026
Status	Submitted for approval
Decision requested by	30 September 2026
Live build	kaonhew02.github.io/MiniShoppingMall
Repository	github.com/KaonHew02/MiniShoppingMall

THE ASK

Approval for a ten-week Phase 4 — the seventh shop, sound, collision, a late-game balance pass and a v1.0 release — with one part-time developer (~100 hours) and RM 0 committed.

Document control

Document	Mini Shopping Mall — Project Proposal
Revision	2, 23 September 2026
Author	Kaon
Status	Submitted for approval — decision requested by 30 September 2026
Baseline	Repository <code>KaonHew02/MiniShoppingMall</code> at commit <code>d4c0d1f</code> (23 September 2026)
Source of truth	<code>docs/PROPOSAL.md</code> in the repository. This document is generated from it by <code>npm run proposal</code> , so the two never disagree

Revision history

Rev	Date	Summary
1	22 Sep 2026	First issue — the game specification as built: core loop, shop roster, per-shop specification, staff and logistics, economy, rulebook, interface guide, art direction, architecture, save data, roadmap and risks
2	23 Sep 2026	Reorganised as a full project proposal — background, solution, objectives, players, testing, plan, budget, sign-off and glossary added; new Security and anti-cheat section after the 23 September hardening; architecture, save shape and risks updated for the release build and the seal; mall-level formula corrected; diagrams and screenshots; Word and PDF editions

How to read this proposal

Sections 1–5 make the case: what the game is, the gap it answers, and how success is measured. Sections 6–13 are the game itself — the loop, the six shops, staff, the economy, the rulebook, the interface and the art — written so the game could be rebuilt from them without the source. Sections 14–17 are how it is built, saved, protected and tested. Sections 18–22 are the proposal proper: the plan, the risks, the budget, what comes after, and the decision. Section 23 is a glossary.

Contents

1 Executive summary	5
2 Background and problem statement	6
2.1 What the alternatives get wrong	6
2.2 The design flaw underneath all of it	6
2.3 Where the project started	6
3 Proposed solution	7
3.1 The genre is arcade-idle, not idle-clicker	7
3.2 Five design pillars	7
3.3 One new scarce resource per shop	7
3.4 What the player gets that they did not have	7
4 Objectives and success criteria	8
4.1 What "done" means for Phase 4	8
4.2 Explicit non-objectives	9
5 Target players and platform	10
5.1 Jobs to be done	10
5.2 Audience and platform	10
5.3 What it is not	10
6 Game specification	11
6.1 The core loop	11
6.2 World and projection	11
6.3 Movement and physical constants	11
6.4 Tick model	12
6.5 Shopper arrival and the escalator	12
6.6 The shopping list	12
6.7 Systems every shop shares	13
7 Shop roster and progression	14
7.1 The mall is a ring	15
7.2 Why each shop is a different game	15
7.3 Product lines within a shop	15
8 Per-shop specification	17
8.1 Grocery Store — the stocking game	17
8.2 Coffee Shop — the service game	17
8.3 Burger Rush — the bottleneck game	18
8.4 Sport Outlet — the selling game	19
8.5 Fashion Boutique — the matching game	19
8.6 TechHub — the comparison game	20
9 Staff, stock and logistics	21
9.1 The roster	21
9.2 What a stocker actually does	21
9.3 Capacities	21
9.4 Earning while you are away	22
9.5 Restock and production timing	22

10 Economy and balance.....	23
10.1 Price and upgrade curves	23
10.2 Mall level, gems and the boost.....	23
10.3 Starting position and the first purchases.....	24
10.4 Patience, by shop.....	24
10.5 The buy roll.....	24
10.6 Failure states	25
10.7 The known balance lesson.....	25
11 Rules of play	26
11.1 Opening a shop	26
11.2 Legal actions	26
11.3 Carrying.....	27
11.4 Serving at the till.....	27
11.5 Scoring	27
11.6 Fair play	27
11.7 Edge cases.....	28
12 Interface guide.....	29
12.1 Screen inventory.....	29
12.2 HUD anatomy	30
12.3 The dock	30
12.4 Controls	30
12.5 Navigation flow	31
12.6 The bottom sheet	31
12.7 Products sheet.....	31
12.8 Staff sheet.....	32
12.9 Map sheet.....	32
12.10 Boost sheet.....	32
12.11 Settings sheet	32
12.12 Offline sheet	32
12.13 In-world feedback.....	33
12.14 Notification language.....	33
13 Art direction, brand and audio.....	34
13.1 Identity	34
13.2 The face shading rule.....	34
13.3 Palette.....	34
13.4 Product painters	34
13.5 Logo pipeline	35
13.6 Audio.....	35
14 Technical architecture	36
14.1 Stack	36
14.2 File layout	37
14.3 The shop module contract.....	37
14.4 Floor plans	38
14.5 Movement and collision	38
14.6 Rendering	38
14.7 Deployment	38
15 Save data and persistence	40

15.1 Shape	40
15.2 The save-index rule.....	40
15.3 Forward-compatible merging	41
15.4 Export, import and Drive	41
16 Security and anti-cheat.....	42
16.1 Threat model	42
16.2 Controls in place	42
16.3 Privacy by construction.....	43
16.4 What is not protected, and why	44
17 Testing and quality assurance	45
17.1 Measured results on record	45
17.2 The invariant worth testing	45
17.3 Browser matrix	46
17.4 Per-floor acceptance.....	46
17.5 Per-shop acceptance	46
17.6 Economy acceptance.....	46
17.7 Release checklist.....	47
18 Project plan and timeline	48
18.1 Phases 1–3 — delivered.....	48
18.2 Phase 4 — proposed.....	48
18.3 Milestones	49
19 Risks and mitigations	50
19.1 The three that deserve a decision, not just a mitigation	50
20 Resources and budget	52
20.1 People	52
20.2 Tools and infrastructure	52
20.3 Optional and conditional costs	52
20.4 Three-year total cost of ownership	53
20.5 What the budget does not buy.....	53
21 Future roadmap	54
21.1 Next: shop 4, the Pizza Restaurant	54
21.2 Deferred by shop	54
21.3 Cross-cutting candidates	55
21.4 Deliberately not built, and why	55
21.5 Open questions.....	55
21.6 The sibling apps	55
22 Conclusion and approval	56
22.1 The decision requested.....	56
22.2 Sign-off.....	56
22.3 Sources	56
23 Glossary.....	58

1 Executive summary

Mini Shopping Mall is a browser-based arcade-idle store tycoon. You drive one character round one shop floor at a time, read what each customer wants from the bubble over their head, do the work they are waiting on, and take the money off the floor. Earn enough and the escalator in the corner opens the next shop — which asks a different question. Six of the seven planned shops are built, each a different game sharing one character, one economy and one save file. A working build is live at kaonhew02.github.io/MiniShoppingMall.

The pitch: *you are the whole staff of a mall, one unit at a time.*

Product	Mini Shopping Mall — <i>Idle Tycoon</i>
Genre	Arcade-idle store tycoon, single player
Platform	One HTML page — phone, tablet or desktop browser; touch-first, joystick-driven
Delivered	6 of 7 shops playable; four languages; save, backup and Google Drive copy
Code	13,955 lines of plain JavaScript in 24 files, plus one stylesheet and one page
Security	Release build sealed in one closure; saves sealed with a hash; Content-Security-Policy
Dependencies	None at runtime. Two build tools (esbuild, resvg) and no framework
Running cost	RM 0/month — GitHub Pages hosting, the player's own browser for storage
Effort to date	29 commits, 27 Aug 2026 – 23 Sep 2026
Proposed next phase	10 weeks: the seventh shop, sound, collision, late-game balance, a store-ready v1.0

The design bet is that one control scheme can carry six unrelated genres. Each shop keeps the joystick and the thought bubble and changes what the bubble means:

Shop	Unlock	The question it asks	Genre
Grocery Store	free	Is the shelf full?	stocking
Coffee Shop	\$6K	Is it made yet?	service
Burger Rush	\$45K	Which station is holding up the tray?	bottleneck
Sport Outlet	\$180K	Have they tried it, and did anyone advise them?	selling
Fashion Boutique	\$4.2M	Have you got it in their size, and is a cubicle free?	matching
TechHub	\$95M	Which of the two is right for them?	comparison

This document is two things at once. Sections 6 to 13 are the full game specification — the loop, every shop, the staff, the economy, the rulebook and a screen-by-screen interface guide — so the game can be handed to an artist, a balancer or a second engineer without reading the source. The rest is the proposal proper: what the game is for, how it is built and protected, how it is tested, what could go wrong, and what the next phase costs.

The ask. Approval to run the ten-week Phase 4 in [Project plan and timeline \(\\$18\)](#) at the resourcing set out in [Resources and budget \(\\$20\)](#) — one developer, part-time, about 100 hours, and RM 0 committed.

2 Background and problem statement

The arcade-idle genre — you walk a character round a shop, carry stock, and hire staff to automate what you have grown bored of — is one of the most-played casual formats on phones. Its best-known games prove the loop works. Most of them also stop asking anything new after the first shop.

2.1 What the alternatives get wrong

Option	What it costs the player
Tap-to-collect idle clickers	No body in the game. The only verb is a tap on a menu, and the only decision is which number to raise next
Arcade-idle clones	The first shop is a good game. The second is the same shop re-skinned — a customer walks in, picks something up and pays — so the player has learned everything in ten minutes
Free-to-play tycoon apps	Energy timers, forced adverts and gem stores sit between the player and the loop. The game is tuned to sell, not to play
Deep management simulations	Rewarding, but built for a desk and an hour, not a phone and five minutes on a train

2.2 The design flaw underneath all of it

Most arcade-idle games scale by **number**, not by **question**. Shop two sells dearer goods faster; shop five sells dearer goods faster still. The player's hands do the same thing for the whole game and only the price tag changes. The brief for this project was written against exactly that: *do not make seven shops where a customer walks in, clicks and pays*.

2.3 Where the project started

Development began on 27 August 2026 with the Grocery Store, the isometric engine and the save layer on the first day. An earlier tap-a-shop-to-collect prototype was discarded for the reason in the table above — same art, wrong product. By 4 September six shops, four languages, the brand kit, the save layer and the Drive copy had landed; revision 1 of this proposal followed on 22 September, and the security hardening on 23 September.

3 Proposed solution

A mall of shops that share a body, a currency and a save file — and nothing else. Every shop keeps the joystick and the thought bubble, and changes what the bubble means and what the player's hands must do about it.

3.1 The genre is arcade-idle, not idle-clicker

The player's body is in the game: you walk, you carry, you stand at the till. Staff exist to automate the parts you have grown bored of, which means the manual loop has to be worth doing first.

3.2 Five design pillars

Every feature is checked against these before it is built.

1. **The player's body does the work.** Before designing anything, ask where the character stands and what they carry. A mechanic that resolves in a menu is not a mechanic.
2. **Every shop is its own game.** Each shop must change the question, not the skin.
3. **Nothing is delivered — it is made, fetched or fitted on site.** The chain from raw thing to sale is always walkable, and always visible on the floor.
4. **Automation is a body, not a boolean.** A hire must appear on the floor and be seen doing the job. A hire that is only a flag in a menu reads as a missing character.
5. **Every screen says what to do next.** A locked row with no cost and no position in the order reads as a bug. So does a shop with no guidance line.

3.3 One new scarce resource per shop

Each shop introduces exactly one new thing the player is short of, and that is what they manage. The full table is in [Why each shop is a different game \(§7.2\)](#).

3.4 What the player gets that they did not have

- **Six games for one set of controls.** Stocking, service, bottleneck, selling, matching and comparison — learnt one at a time, never re-learned.
- **No energy timers, no adverts, no store.** Gems are earned from mall levels and nothing sells them.
- **Nothing to install.** A web page that opens on a phone, from a link or from a double-clicked file.
- **Progress that stays theirs.** The save lives in their own browser, and exports to a file or their own Google Drive.

4 Objectives and success criteria

The project succeeds if a player can be dropped into any shop, cold, and know what to do within five seconds — and if all seven shops play as different games on one set of controls. Everything below is a test, not an aspiration.

#	Objective	Measure	Threshold	Status
O1	Each shop is a different game	Shops whose sale depends on a mechanic no earlier shop has	7 of 7	6 of 7 — the Pizza Restaurant is Phase 4
O2	One control scheme carries every shop	Verbs beyond <i>move</i> and <i>stand near</i>	0	Met
O3	Guidance is never missing	Moments the objective line is blank while something useful can be done	0	Met in playtest
O4	Automation is visible	Hires that do not appear on the floor as a body	0	Met
O5	Hiring pays	Shops where the full crew earns less than the empty floor	0	Met — TechHub 70% with an advisor vs 62% without
O6	Floors are walkable	Gaps narrower than 1.0 unit on a walking route	0	Met on all six floors
O7	Runs with nothing installed	Servers, accounts or plug-ins needed to play	0	Met — even <code>dist/index.html</code> opens by double-click
O8	Reaches its players in their language	Languages with every interface string	4	Met — English, 简体中文, 繁體中文, Bahasa Melayu
O9	A reload loses nothing that was paid for	Half-paid builds and levels lost across a reload	0	Met
O10	Cash cannot be typed in from the console	Game state reachable from <code>window</code> in the live build	0	Met (23 Sep 2026)
O11	An edited save is refused	Hand-edited saves or backup files accepted	0 after 1 Nov 2026	Met — legacy window until 1 Nov
O12	A deploy is never served stale	Script files served from an old cache after a release	0	Met — content-hashed file names
O13	The late game is balanced	Unlock tiers actually played through	6 of 6	Not met — nobody has played to \$95M
O14	The game has sound	Feedback cues with an audio cue	8 of 8	Not met — no engine yet

4.1 What "done" means for Phase 4

O1, O13 and O14 are the three open rows, and they are the whole of the proposed next phase: the seventh shop, a balance pass that plays the late game for real, and a sound engine.

4.2 Explicit non-objectives

These are refused on purpose, and each refusal has a reason:

- **No energy, no adverts, no paid gems.** They put a wall between the player and the loop, and the loop is the product.
- **No placement mode.** The player never places a fixture; every floor is authored. Free placement is a city builder, and it would break the walking-room rules in [Game specification \(§6\)](#).
- **No multiplayer, no leaderboard.** A leaderboard would need a server to be honest, and a server is exactly what the security model can do without.
- **No tap-to-collect.** It was built, and discarded — see [Background and problem statement \(§2\)](#).

5 Target players and platform

The target is a mobile-first casual player who knows *My Mini Mart*, *Idle Mall Tycoon* or *Burger Please* — the three stated references — and plays in sessions of two to fifteen minutes.

Persona	Situation	What they need	Lives in
Aina, 22 — the commuter	Twenty minutes on the LRT, one thumb free	A loop that is readable in a glance and pays for idling	The joystick, the objective line, cash that banks itself
Wei Jie, 29 — the optimiser	Enjoys a puzzle more than a grind	A bottleneck that moves when you fix it, and a number that proves it	Burger Rush, the Staff sheet's conversion figures
Priya, 17 — the collector	Wants to see every shop open	A visible ladder of what comes next, and when	The Map ring, dock badges, unlock prices
Encik Rahman, 44 — the returner	Opens it once a day	Something earned while away, and one clear next step	Offline earnings, the guidance line

One player is usually several of these at once, which is why the shops scale from a two-minute Grocery session to TechHub's comparisons.

5.1 Jobs to be done

- *When I have two minutes, I want to know what to do the moment the game opens, so that the session is not spent reading.*
- *When I come back tomorrow, I want the mall to have earned something, so that coming back feels worth it.*
- *When a shop stops being interesting to do by hand, I want to hire someone to do it, and see them do it, so that the hire feels like progress.*
- *When I unlock a new shop, I want to learn something new, so that it is not the last shop again.*

5.2 Audience and platform

It ships as a single HTML page: touch-first, joystick-driven, portrait-friendly, and playable from a double-clicked [index.html](#) with no server. That constraint is deliberate — it keeps the game distributable as a folder, a static host or a wrapped mobile shell without changing a line. Four languages are shipped: English, Simplified Chinese, Traditional Chinese and Bahasa Melayu.

5.3 What it is not

Not a clicker. Not a city builder — the player never places a fixture; every floor plan is authored. Not multiplayer, and not online: there is no server, no account and no leaderboard.

6 Game specification

The player occupies exactly one shop at a time. Every shop is a closed, authored floor with a street door, a counter, a back-of-house area and an escalator. The other shops keep running while you are away, but only if they are staffed.

6.1 The core loop



Figure 1 — The core loop — the gold step is the only one that differs between shops

The gold step — *you do the work* — is the only one that differs between shops. Everything around it is shared.

6.2 World and projection

The scene is a 2:1 dimetric isometric grid drawn to a single canvas. One cell is 64 px wide by 35 px tall; one unit of height is 42 px. Tile size is fitted to the smaller screen axis, so a wide desktop window does not zoom past the floor.

The camera trails the player rather than being dragged, easing toward a point 0.6 units above their feet. It is clamped so all four screen corners stay on the floor at any zoom between 0.7x and 1.7x.

Each shop declares its own floor box. The Grocery Store is the largest at 27.5 x 19.0 units, because it is the only one with a farm bolted onto the back; the default box is 23.5 x 15.2.

6.3 Movement and physical constants

Constant	Value	Meaning
PLAYER_SPEED	3.8 units/s	top speed at full stick
STAFF_SPEED	2.9 units/s	hired staff
CUSTOMER_SPEED	2.1 units/s	shoppers
ACCEL	14	how sharply a new heading is taken up
BODY_R	0.22 units	body radius, for separation and walls
REACH	0.8 units	how close you stand to use a station
CARRY_CAP	12 items	arms hold any mix, stacked overhead
CARRY_SLOW	0.4	fraction of speed lost with full arms
HANDLE_RATE	9 items/s	in and out of your arms

Only the player collides with fixtures. Customers and staff collide with each other but pass through scenery, which is a known limitation.

6.4 Tick model

The game runs one `requestAnimationFrame` loop. Frame delta is clamped to 0.1 s; a delta over 1.5 s (a backgrounded tab) is handed to a catch-up pass and the frame proceeds at 0.05 s. Any Rush Hour counts down by that same delta, so a boost runs on game time. Order within a frame is fixed:

1. Rebuild the walkability cache, then run station restock timers.
2. Move the player, the stockers and the cashier.
3. Spawn and step customers; age cash piles.
4. Run the active shop module (cafe, food, sports, boutique or tech); the generic till runs only for shops that do not own their counter.
5. Separate bodies so no two end the frame in the same spot.
6. Run the floor pads in order: till plot, build plots, OPEN sign, guidance, level pads, doors, passive income from other shops, mall level check.
7. Ease the camera, render, refresh the HUD.

Progress autosaves every 10 seconds of loop time, and on `visibilitychange` and `pagehide`.

6.5 Shopper arrival and the escalator

Customers arrive only while the OPEN sign is flipped. The gap between arrivals interpolates from 3.0 s when quiet to 1.3 s when busy, driven by mall level over 12, and the floor is capped at 9 shoppers at once.

Once a second shop is open, 45% of arrivals and departures use the escalator instead of the street door, because those shoppers are doing the whole mall rather than one unit. A ride takes 2.6 s end to end; the run climbs 1.05 units and the rider's height tracks the tread exactly. With one shop open the escalator is idle.

6.6 The shopping list

Outside the tutorial, a Grocery shopper carries a list rather than a single want.

Roll	Odds
1 / 2 / 3 different products	45% / 33% / 22%
1 / 2 / 3 of each product	55% / 28% / 17%
Total items carried	capped at 6

They collect shelf by shelf, taking 0.28 s to lift each item, then pay one total at the till. A shopper who hovers behind a full queue for 25 seconds gives up and puts the whole basket back.

6.7 Systems every shop shares

- **Build plots.** A product line, a machine, a court, a cubicle or the till itself starts as a plot on the floor. Stand on it and your cash drains in until it is paid; partial payment is remembered.
- **Level pads.** Every station has a pad beside it. Standing on it pours cash into the next level, raising that line's price and its production speed.
- **The OPEN sign.** By the door. Flipping it is what lets customers in.
- **Cash on the floor.** Each sale drops a pile. Walk over it to bank it; a pile left 9 seconds banks itself, so idling still pays.
- **The bin.** Walk into it to empty your arms when you picked up the wrong thing.
- **The queue.** A counter with a queue slot line running back toward the door.
- **Passive income.** Every shop you are *not* standing in accrues its rate continuously, provided it is staffed.

7 Shop roster and progression

Six of the seven planned shops are built. The seventh, a Pizza Restaurant, is specified but not implemented; it is the only gap in the roster.

#	Shop	Signature loop	Unlock	Status
1	Grocery Store	Stock → Pick → Checkout	free	Delivered
2	Coffee Shop	Order → Prepare → Serve → Table	\$6K	Delivered
3	Burger Rush	Order → Kitchen → Assemble → Pickup	\$45K	Delivered
4	Pizza Restaurant	Dough → Toppings → Bake → Box	to be set	Open — Phase 4
5	Sport Outlet	Browse → Try → Advice → Buy	\$180K	Delivered
6	Fashion Boutique	Browse → Size → Fitting room → Buy	\$4.2M	Delivered
7	TechHub	Browse → Demo → Compare → Buy	\$95M	Delivered



Figure 2 — The six built shops, captured from the running game

The unlock ladder is steep on purpose: each shop is roughly 4x to 20x the last, and the two late shops are separated by more than an order of magnitude, because TechHub's products are worth millions each.

7.1 The mall is a ring

Shops are not a menu. Each floor has an escalator in one corner that rides to the next shop, and the last one wraps back to the first. The Map draws that ring directly.

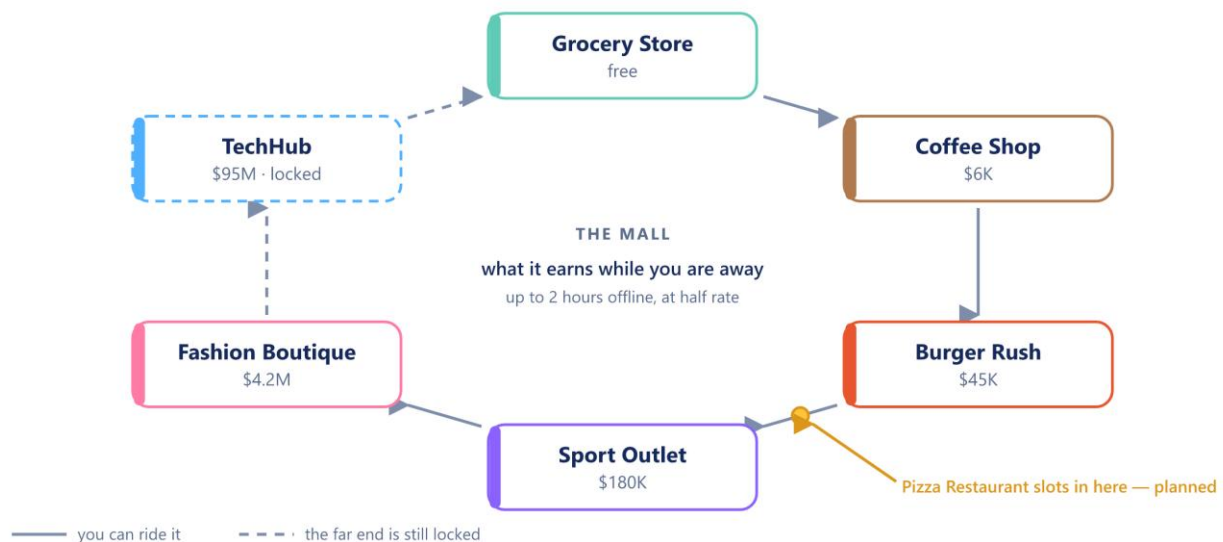


Figure 3 — The mall ring, as the Map draws it mid-game with TechHub still locked

A link is drawn solid where you can ride it and dashed where the far end is still locked. Tapping a box travels there, or buys it open if you can afford it. The middle of the ring carries the one figure that belongs to the mall rather than to any shop: what the place earns while you are away.

You can also travel physically. Stand in front of the escalator and hold for a moment, and it carries you up.

7.2 Why each shop is a different game

Each shop introduces exactly one new scarce resource, and that resource is what the player manages.

Shop	The scarce thing	The failure it produces
Grocery	shelf stock, and the feed that makes it	an empty shelf; the shopper walks
Coffee	machine time, ingredients in the bar, clean tables	a cold queue and no tips
Burger Rush	the slowest station's throughput	tickets paid for and refunded
Sport Outlet	your attention — someone must advise	a shopper who tried it and left it
Boutique	a size on the rail, and a free cubicle	a full rail that is still empty to them
TechHub	sealed boxes, and a translated spec sheet	a convinced buyer at an empty stand

7.3 Product lines within a shop

Every shop opens its lines one at a time, in a fixed order, at their own build plot on the floor. Rank sets the price band, so later lines are always worth more. The Grocery Store has 18 lines (17 sellable plus wheat, which is feed); the Coffee Shop has 16 (six ingredients and ten recipes); the other four have 8 each.

The cheapest line in each shop is free and arrives with the store. The dearest is a long-term goal: oranges at \$85K in Grocery, a TechVision TV at \$260M in TechHub.

8 Per-shop specification

Each shop below is given as: its floor, its lines, its stations, its staff, and the one rule that defines it.

8.1 Grocery Store — the stocking game

Floor 27.5 x 19.0, the largest in the mall, because nothing is delivered. Nine crop beds run across the back wall, an orchard of three trees down the right, and the farmyard is a column down the left, ordered so nothing is far from what it eats: pig, chicken, wheat field, cow, oven, yogurt vat. The shop floor in the middle holds the shelves in three rows, tinted and labelled by department.

The production chain is the shop:

Product	Comes from	Needs
Potato, Tomato, Carrot, Eggplant, Cabbage, Cucumber, Watermelon, Strawberry, Blueberry	crop beds	time only
Apple, Banana, Orange	the orchard	time only
Wheat (not sold)	the wheat field	time only
Milk	the cow	wheat in the trough
Bread	the oven	wheat
Yogurt	the vat	milk
Bacon	the pig pen	potatoes
Eggs	the chicken coop	tomatoes

The wheat field sits between the cow and the oven — the only two things that want it — and the vat sits under the cow it draws milk from. That placement is a rule, not a decoration: an earlier layout put wheat 19 units from the trough and it played badly.

Prices run from \$9 (potato) to \$355 (orange). Lines unlock from free (potato) to \$85,000 (orange) in 18 steps.

The defining rule: a shelf holds 8. A shopper standing at an empty shelf is the only thing that drains patience.

8.2 Coffee Shop — the service game

Nothing waits on a shelf. Six ingredients live in crates along the back wall and must be carried to the **ingredient storage** beside the bar; the machines draw only on that storage, never on the crates. An empty bin is why nothing is brewing.

Ten recipes live on a menu board, each its own build plaque. Three machines make them:

Machine	Build cost	Worked by	Makes
Coffee Machine	free	Barista	Espresso, Americano, Latte, Cappuccino, Iced Coffee, Mocha
Matcha & Ice Bar	\$4,000	Barista	Matcha Latte, Iced Matcha
Pastry Oven	\$12,000	Chef	Croissant, Cake

Prices run \$90 (espresso) to \$560 (cake). Once the oven is built, 45% of tickets add a food item and a further 15% add a second; the ticket splits one job per item, each routed to its own station, and the customer pays the whole bill when the last item reaches their hands.

Seating is six tables, two free and four build plots at \$900, \$2,400, \$6,000 and \$14,000. A served customer sits for 8 to 15 seconds and leaves the table dirty. A dirty table cannot be sat at and drags tips down; wiping one takes 1.1 s. If every table is dirty or taken, they take the drink away instead.

Four jobs beyond runner and cashier: Barista, Chef, Server, Cleaner. A machine runs only when its own person is hired, or you stand at it. The barista never bakes and the chef never brews.

The defining rule: tips, capped at 25% of the price, are where the money is — and they are paid for speed in a clean room.

8.3 Burger Rush — the bottleneck game

Eight lines across three stations, and every combo touches all three. The ticket splits three ways, the parts cook in parallel, and the tray is not assembled until the slowest part is done.

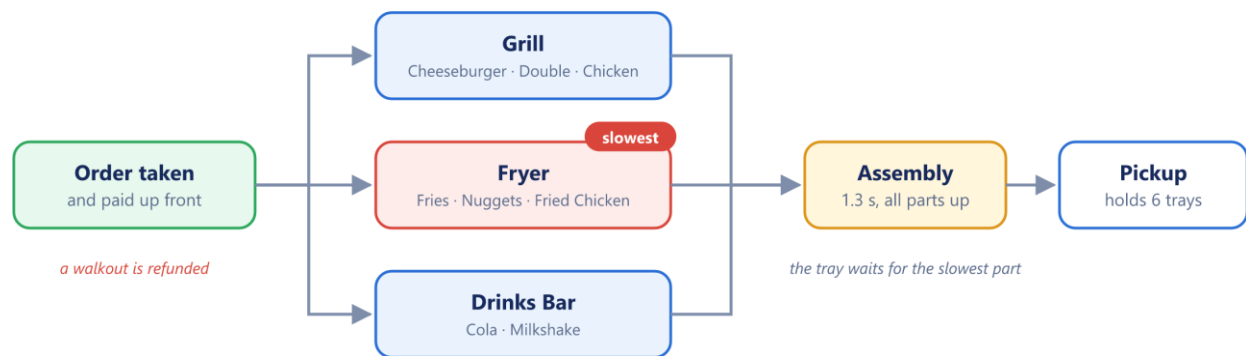


Figure 4 — Burger Rush — three stations in parallel; the tray waits for the slowest

Station	Build cost	Lines
Grill	free	Cheeseburger \$190, Double Burger \$320, Chicken Burger \$480
Fryer	\$5,000	Fries \$120, Nuggets \$210, Fried Chicken \$640
Drinks Bar	\$14,000	Cola \$100, Milkshake \$260

62% of customers want the meal rather than just the main. Each station shows its own load on the floor and the worst is flagged red. Levelling a station does not end the problem, it moves it: a measured run took the fryer from load 3.7 to 0.4 and the grill from 1.2 to 3.0, and trays out went from 39 to 44 over the same 80 seconds.

This is the only shop that takes money **up front**, which means a walkout must be **refunded**. Without the refund a jammed kitchen would be free money and the bottleneck would stop mattering. With nobody hired the line seizes: a measured run gave 15 orders taken, 0 trays out, 9 walkouts.

Each line has a **line bin** of raw stock in front of its station. A station with an empty bin cannot start however fast it is. Two hires: a **Line Cook** works the stations, a **Packer** builds the trays.

The defining rule: the shop runs at the speed of its worst station, never its best.

8.4 Sport Outlet — the selling game

The goods are already made and already on the rack. Four sports, two lines each, and a court for every one of them.

Sport	Lines	Court	Court cost
Running	Running Shoes \$2.4K, Water Bottle \$3.2K	treadmill	free
Football	Football \$4.8K, Football Boots \$6.4K	goal	\$60,000
Basketball	Basketball \$9K, Team Jersey \$12K	hoop	\$200,000
Badminton	Racket \$17K, Shuttlecocks \$24K	net	\$500,000

A sport without a built court still sells, to about half as many people. A shopper picks a product up, carries it to its court, tries it for 3 to 5 seconds, then decides: buy, too dear, or not for me. A rejection goes back on the rack, and half the time becomes "show me another one".

Advice is the other half of the sale. A shopper holding something and wondering shows a question mark. Stand within 1.6 units for 0.9 s and they are talked into it. Good advice also raises what they will pay by 25%, which rescues shoppers who simply cannot afford what they picked up. A hired **Sports Advisor** does this without you, and the shop earns nothing unattended until you have one.

The defining rule: nobody buys what they have not tried.

8.5 Fashion Boutique — the matching game

Eight lines in four departments. Six are garments, which carry four sizes and must be tried on; the cap and the handbag have no size and skip the cubicle, which makes them the quick sale when the shop is heaving.

Every rail shows its size breakdown on the floor — **S 2 · M 0 · L 1 · XL 3** — with a red pip for a size that has run out. Shoppers arrive wanting S, M, L or XL at 20 / 34 / 30 / 16 percent, so the middle sizes empty first. That is the whole reason the stockroom ever gets a visit.

A shopper at a rail with none of their size raises a ruler and waits on a **shorter fuse than anything else in the game** — 45 s against the shop's normal 70. Walk to the stockroom, pick one up, carry it over, and it goes into their hands; being served like that is worth a sale on its own. A hired **Fashion Assistant** runs those errands.

Five cubicles: two free, then \$120K, \$400K and \$900K. They are the bottleneck.

Fitting rooms	Conversion
2	41%
5	66%

Prices run \$38K (cap) to \$380K (handbag). 42% of shoppers are after two pieces rather than one.

The defining rule: a full rail can still be an empty rail.

8.6 TechHub — the comparison game

Nobody comes in for a named product. They come in for a *kind* of thing, with a priority and a budget, and every department stocks a pair that pulls in opposite directions.

Department	The pair	The argument	Demo bench
Audio	TechBuds \$900K vs TechSound Max \$1.9M	battery vs room-filling sound	free
Phones	TechPhone \$1.4M vs TechPhone Pro \$2.6M	all-week battery vs the camera	\$30M
Laptops	TechBook Air \$3.4M vs TechBook Pro \$6M	thin and long-lived vs fast and hungry	\$90M
Screens	TechView 144 \$4.5M vs TechVision TV \$8M	refresh rate vs sheer size	\$220M

Products carry rated specs — performance, battery, camera, display, sound — on a 1-to-5 scale, and the customer's own priority decides which of the pair is "better". They look at the first, demo it, look at the second, demo that, weigh the two, and buy the winner they can afford.

Demoing is free; selling needs a box. Fifty people can try the floor unit, but a sale takes a sealed box on the stand — that is what the stockers haul. A shopper who has already decided will wait at an empty stand exactly as long as their patience lasts.

Shop floor	Conversion
Cold — no benches, nobody advising	28%
All four benches	62%
Benches plus a Tech Advisor	70%

The defining rule: a spec sheet is noise until somebody translates it.

9 Staff, stock and logistics

Staff are bought once, not paid wages. Every hire appears on the floor as a body doing the job, which is a hard requirement: a hire that is only a flag in a menu reads as a missing character rather than an upgrade.

9.1 The roster

Role	Shops	Cost formula	Floor
Stocker (runner)	all	$\max(1800, \text{unlock} \times 0.7) \times 3.2^{\text{owned}}$	\$1,800
Cashier	all	$\max(2500, \text{unlock} \times 0.9)$	\$2,500
Barista	Coffee	$\max(3200, \text{unlock} \times 1.1)$	\$3,200
Chef	Coffee	$\max(4000, \text{unlock} \times 1.35)$	\$4,000
Server	Coffee	$\max(4800, \text{unlock} \times 1.6)$	\$4,800
Cleaner	Coffee	$\max(2200, \text{unlock} \times 0.75)$	\$2,200
Line Cook	Burger Rush	$\max(2600, \text{unlock} \times 0.9)$	\$2,600
Packer	Burger Rush	$\max(3400, \text{unlock} \times 1.2)$	\$3,400
Sports Advisor	Sport Outlet	$\max(9000, \text{unlock} \times 1.4)$	\$9,000
Fashion Assistant	Boutique	$\max(12000, \text{unlock} \times 1.3)$	\$12,000
Tech Advisor	TechHub	$\max(30000, \text{unlock} \times 1.2)$	\$30,000

unlock is the shop's own purchase price, so staff scale with the shop they work in. Stockers are capped at four per shop and each costs 3.2x the last, which makes the fourth roughly 33x the first.

9.2 What a stocker actually does

A stocker runs a two-priority job queue. Feed comes first: it hauls input to whatever station is starving — wheat to the cow, potatoes to the pig, tomatoes to the chicken, milk to the vat. Only then does it restock the emptiest shelf.

Stocker **claim** jobs, so two never chase the same shelf. One alone cannot keep eleven shelves and four feed stations running, which is the reason the cap is four rather than one.

9.3 Capacities

Store	Holds
Your arms	12 items, any mix
A shelf	8
A station's output crate	10 finished items, then it stalls
An animal or machine's feed hopper	8 input items
Cafe pickup counter	8 finished drinks
Burger Rush pickup counter	6 trays

A station whose output crate is full stops producing. That is the signal to go and empty it.

9.4 Earning while you are away

A shop accrues income continuously while you stand in a different one, and offline for up to 2 hours at 50% rate. It earns **nothing** unless every gate below is satisfied:

Gate	Applies to
Owned, till built, sign OPEN	every shop
At least one stocker, and a cashier	every shop
Barista and Server hired	Coffee Shop
Chef hired — otherwise the food lines alone earn nothing	Coffee Shop
Line Cook and Packer hired	Burger Rush
Sports Advisor hired	Sport Outlet
Fashion Assistant hired	Boutique
Tech Advisor hired	TechHub

The unattended conversion rate is the shop's own buy formula with the advice bonus applied and the demo or fitting bonus applied where the facility is built, then multiplied by 0.85 — because some shoppers still cannot afford it. Unattended earning is therefore always slightly worse than a good human.

9.5 Restock and production timing

Each line declares a **restock** period in seconds, from 1.2 s (ice) to 3.4 s (orange, bread). A station produces one item per period, scaled by the speed multiplier its level has earned. **MIN_RESTOCK** floors the period at 0.35 s, so no amount of levelling makes a station instant.

Machine and station levels buy two things at once. In the cafe, each level adds 0.34 to brew speed and every third level adds a concurrent cup, capped at 4. In Burger Rush, each level adds 0.30 to cook speed and every second level adds a concurrent part, capped at 5 — the faster ramp is what lets the player chase a moving bottleneck.

10 Economy and balance

Everything tunable lives in one file, `src/config.js`. No other file hard-codes a number, and that is enforced by convention rather than by tooling.

10.1 Price and upgrade curves

A line's sale price is linear in its level, stepped by milestone multipliers and by any active boost:

```
price    = base × level × M_income(level) × B
cost(k)  = base_up × 1.13^(level - 1)      base_up = max(60, base price × 7)
```

The cost of the next level grows geometrically at 1.13 per level. Levels are capped at 100. Milestones multiply price and production speed on the way up:

Level	Income multiplier	Speed multiplier
10	x2	—
25	x2	x2
50	x2	x2
100	x3	x2

A fifth milestone at level 200 sits above the cap and can no longer be reached. It is kept in the table so that a save made before the cap existed keeps the multipliers it already bought — capping levels must not quietly nerf someone's shop.

A fully levelled line therefore earns 24x its base price per item, and produces at 8x its base speed, floored at one item per 0.35 s.

10.2 Mall level, gems and the boost

Mall level rises with the sum of every product level in every shop you own. Reaching mall level n needs a total of $5n(n-1)$ levels: level 2 at 10, level 5 at 100, level 10 at 450. Every line counts from level 1, built or not, so a new game's 18 Grocery lines already put the mall at level 2.

Each mall level pays 3 gems. Fifteen gems buy a Rush Hour: x2 income for 60 seconds, counted down in game time. Mall level also drives demand — arrival spacing interpolates from 3.0 s to 1.3 s as mall level climbs toward 12.

10.3 Starting position and the first purchases

Item	Cost
Starting cash	\$500
Checkout counter	$\max(100, \text{unlock} \times 0.02)$, so \$100 in Grocery
First product line	free
Second line (tomato)	\$250
First stocker	\$1,800
First cashier	\$2,500
Coffee Shop	\$6,000

10.4 Patience, by shop

Patience is the ring around a customer's bubble. What starts it draining differs by shop, and that difference is the balance lever.

Shop	Patience	Grace	Drains while
Grocery	25 s queue patience	—	standing at an empty shelf, or behind a full queue
Coffee	46 s	7 s	queueing, and from the moment the order is taken
Burger Rush	55 s	5 s	holding a paid ticket
Sport Outlet	60 s	6 s	waiting at an empty rack
Boutique	70 s, but 45 s on a size request	6 s	queueing for a cubicle, or waiting on a size
TechHub	60 s	6 s	waiting on a sold-out box

The boutique's 45-second size fuse is deliberately the shortest timer in the game, because fetching a size is the stage's signature job and it has to feel urgent.

10.5 The buy roll

Three shops resolve a sale as a probability rather than a transaction. All three cap at 96%.

Term	Sport Outlet	Boutique	TechHub
Base	34%	46%	30%
Tried it / fitted it / demoed it	+30%	+26%	+18%
Compared against the rival	—	—	+16%
Advised by a person	+26%	+22%	+24%
Per product level	+0.8%	+0.8%	+0.8%
Budget, as a multiple of price	0.72x to 2.20x	0.80x to 2.40x	0.70x to 2.20x
Budget after good advice	x1.25	—	x1.30

A rejected item goes back on the rack. In the Sport Outlet, half of all rejections become "show me another one" rather than a walkout.

10.6 Failure states

Failure	Where	Cost to the player
Walkout at an empty shelf	Grocery	the sale
Abandoned queue after 25 s	Grocery	the whole basket goes back
Ticket expires	Coffee	the sale and the tip
Every table dirty or taken	Coffee	they take it away; tips drop
Ticket expires	Burger Rush	the sale and a refund , because they paid up front
Empty line bin	Burger Rush	the station cannot start at any level
Left unadvised	Sport Outlet, TechHub	conversion collapses toward the base rate
Size missing, nobody fetches	Boutique	a 45-second fuse, then a walkout
No free cubicle	Boutique	conversion falls from 66% to 41%
Sold out of sealed boxes	TechHub	a convinced buyer waits, then leaves

10.7 The known balance lesson

Advice must not replace the rest of the loop. In TechHub, an early build let advised shoppers buy without demoing, and the demo benches stopped mattering. The rule that came out of it: **an advised shopper must still go to the bench**. Advice changes what they buy and what they will pay; it never skips a step of the loop.

11 Rules of play

This section is the rulebook: what the player may do, in what order, and what happens at the edges.

11.1 Opening a shop

Every shop, without exception, opens the same way. A gold arrow on the floor walks the player through it once, and only once, per shop.

1. **Build the counter.** Stand on the checkout plot until your cash has drained in. Cost is 2% of the shop's unlock price, minimum \$100. Nothing can be sold before this.
2. **Get the first line's goods in place.** In Grocery that means harvesting potatoes and putting them on the shelf; in the cafe it means carrying beans to the bar storage; in each shop it is that shop's equivalent.
3. **Flip the OPEN sign** by the door. No customer enters before this.
4. **Serve the first customer** at whatever this shop counts as a counter.
5. **Run over the cash.** First sale. The script ends here and never runs again for that shop.

After the script ends, the same line under the HUD becomes a live to-do, recomputed every frame: the cow whose trough has run dry, a queue with nobody on the till, the next line you can afford to build, the next shop you can afford to buy.

11.2 Legal actions

The player has no verbs beyond moving and standing. Everything is proximity-triggered.

Action	How
Move	Drag anywhere for the floating joystick, or WASD / arrows
Pick up	Walk within 0.8 units of a station with output ready
Put down	Walk to the shelf, hopper or storage that takes it
Empty your arms	Walk into the bin
Serve	Stand behind the till while someone is at the front of the queue
Advise	Stand within 1.6 units of a shopper showing a question mark, for 0.9 s
Hand over a size	Carry it from the stockroom to within 1.4 units of the asker
Build	Stand on the plot; cash drains in continuously
Level up	Stand on the station's level pad; cash drains in continuously
Open or close	Stand on the OPEN sign
Travel	Stand in front of the escalator and hold, or tap a box on the Map
Zoom	Scroll wheel, 0.7x to 1.7x

There is no inventory screen, no drag-and-drop, and no placement mode. If it is not reachable by walking, it is not in the game.

11.3 Carrying

Arms hold 12 items of any mix, stacked over the character's head. A full armful costs 40% of top speed, which is the only movement penalty in the game. Items go in and out at 9 per second, so a full load takes about 1.3 s to transfer.

Carrying the wrong thing is recoverable: the bin empties your arms with no penalty beyond the walk.

11.4 Serving at the till

The front of the queue is served when the player stands at the counter, or automatically when a cashier is hired. Serve time is $\max(0.6 \text{ s}, \text{items} \times 0.35 \text{ s} + 0.25 \text{ s})$ — bigger baskets take longer, because each item gets its beat in the bag.

Standing at the till **while** a cashier is also hired runs the queue 1.7x faster. That is the only place in the game where the player and a hire stack.

11.5 Scoring

There is no score and no end. Four running figures stand in for one:

Figure	Where it shows	What it means
Cash	HUD, top right	spendable now
Gems	HUD, top right	3 per mall level; 15 buys a Rush Hour
Mall level	HUD, centre	sum of every product level you own, against $5n(n-1)$
Conversion	Staff sheet, retail shops	of everyone who walked in, how many walked out with a bag

Conversion is the honest scoreboard in the Sport Outlet, the Boutique and TechHub, because in those shops a customer served badly still leaves without buying.

11.6 Fair play

The game is single player and the only person a cheat can affect is the cheater — but a number typed into a console makes the whole ladder meaningless, so the release build refuses it. The rules are simple: **cash is earned on the floor, and nowhere else**. A save edited outside the game is refused, a button rewritten in the browser's inspector does nothing, and winding the clock back does not replay a boost. How that is enforced is in [Security and anti-cheat \(§16\)](#).

11.7 Edge cases

- **A full output crate stalls its station.** Ten finished items and production stops until you clear it.
- **A full queue turns people away.** Anyone hovering behind it for 25 seconds puts their whole basket back on the shelves.
- **Wheat is not sellable.** It exists only as feed for the cow and the oven, and has no price and no shelf.
- **Accessories skip the cubicle.** The cap and the handbag have no size, so they bypass the boutique's entire bottleneck.
- **A demo never consumes stock.** Fifty people can try the TechHub floor unit; only a sealed box completes a sale.
- **A backgrounded tab does not lose progress.** A frame delta over 1.5 s runs a catch-up pass instead of simulating the gap.
- **Hiding the tab burns a boost.** Time away counts down a Rush Hour exactly as time played does, so it cannot be banked.
- **Losing the pointer mid-drag drops the joystick.** An OS popup, an app switch or a long-press callout used to swallow the pointer-up and leave the character walking into a wall.

12 Interface guide

The whole game is one HTML page: a full-bleed canvas with a fixed HUD over it and one bottom sheet that swaps its contents. There are no other screens, no menus between them and no loading states.

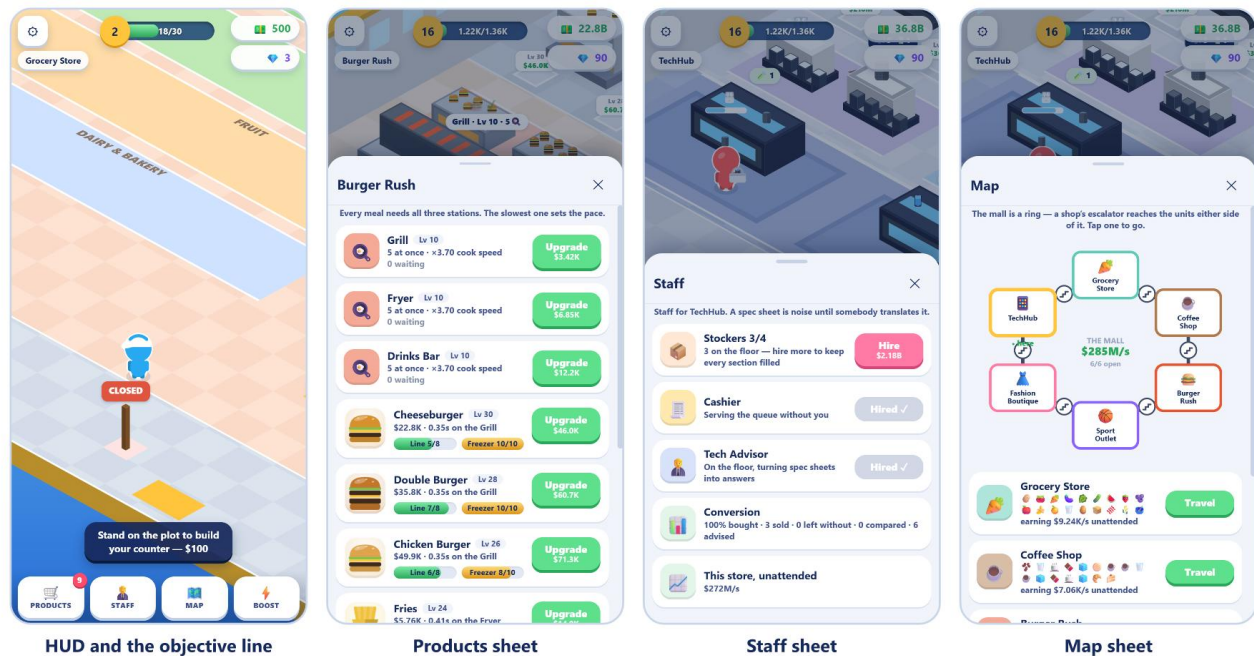


Figure 5 — The interface — a new game's HUD and guidance line, then the Products, Staff and Map sheets

12.1 Screen inventory

Surface	Trigger	Purpose
Canvas	always	the shop floor, the character, everyone in it
HUD header	always	settings, shop name, mall level, cash, gems
Objective line	when there is a next step	one sentence saying what to do now
Carry strip	while holding something	what is in your arms
Boost banner	while a Rush Hour runs	x2 income and the countdown
Dock	always	four buttons: Products, Staff, Map, Boost
Bottom sheet	on any dock or settings tap	the only modal surface in the game
Toasts	on events	1.9 s, bottom, non-blocking

12.2 HUD anatomy

The header is three groups on one row.

Zone	Contents
Left	gear button, then the current shop's name as a tag
Centre	mall level star, a progress track, and have / need in short money notation
Right	a cash pill and a gem pill

Money is abbreviated everywhere: under 1,000 it prints whole, then K, M, B, T, and past that a two-letter suffix (**aa**, **ab**, ...) so TechHub's numbers still fit a pill.

Below the header sit three conditional strips, in this order: the boost banner, the carry strip, and the objective line. The objective line is the single most important element in the interface — it is never empty while there is something useful to do.

12.3 The dock

Four buttons across the bottom, each able to show a badge.

Button	Opens	Badge counts
Products	the shop's line sheet	lines you can afford to level or build
Staff	hires and shop stats	hires you can afford
Map	the mall ring	shops you can afford to buy
Boost	the Rush Hour sheet	—

The badges are the answer to "every screen must say what to do next". A badge on Products means there is money sitting idle.

12.4 Controls

Input	Action
Drag anywhere on the canvas	floating joystick — the stick appears where you touch
Stick travel under 9 px	ignored (dead zone)
Stick travel 46 px	full speed
WASD or arrow keys	the same, at full throttle
Scroll wheel	zoom, 0.7x to 1.7x
Tap a dock button	open that sheet
Tap the scrim or the X	close the sheet

The joystick is analog and floating: a small push walks, a full push runs, and there is no fixed pad to reach for. The stick is dropped on pointer-up, pointer-cancel, lost capture, window blur and tab hide — five paths, because losing any one of them leaves the character walking into a wall.

Movement is also un-projected: "up" on the stick is up the shop, not up the isometric grid.

12.5 Navigation flow

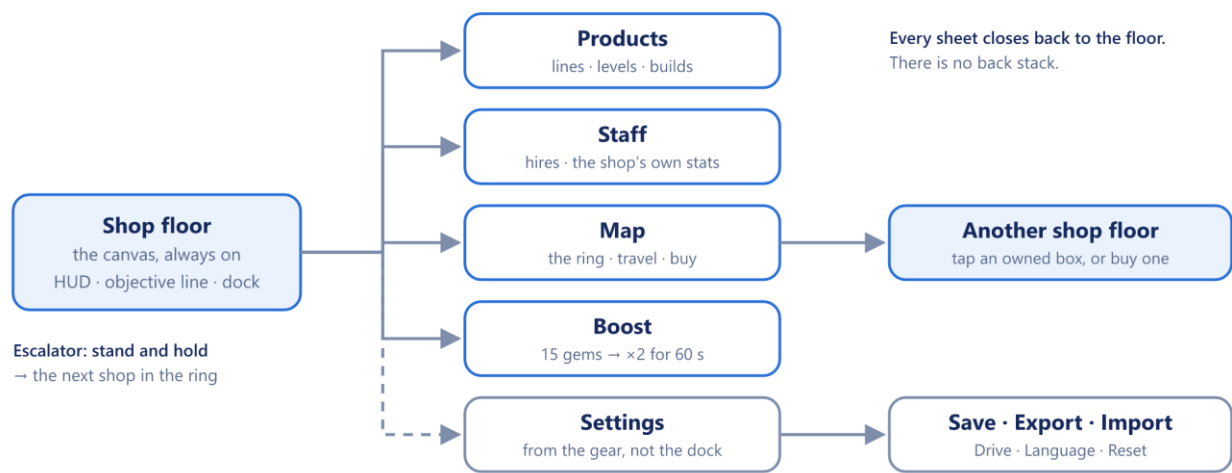


Figure 6 — Navigation — every sheet opens from the floor and closes back to it

Every sheet returns to the canvas. There is no back stack, because there is nothing to go back through.

12.6 The bottom sheet

One sheet element serves every panel. It has a drag grip, a title, a close button and a scrolling body, and it is backed by a scrim that also closes it. The title changes per mode: the Products sheet is titled with the shop's own name, the rest with their function.

The body is rebuilt from a template string on every state change and on a tick, but it is diffed against the last HTML before being written, so scroll position and touch state survive. A click only acts if the sheet actually offered that button — see [Security and anti-cheat \(§16\)](#).

12.7 Products sheet

One row per line, in unlock order. A row is an art chip, a name, a sub-line and an action button.

Row state	What it says
Built	current level, price per item, and an Upgrade button showing the next cost
Built and maxed	a MAX badge instead of a button
Next to build	"UP NEXT" and the build cost
Locked, later	"after <i>Bread</i> — \$2,600", naming the line it is waiting on
Locked, far off	"unlocks later"

Naming the line a locked row is waiting on, and its cost, exists because a bare "unlocks later" with no cost and no place in the order reads as a bug.

Each shop overrides this sheet with its own body. The cafe splits into ingredients, recipes and machines; Burger Rush adds a per-station load figure with the bottleneck flagged; the Boutique prints the size breakdown per rail; TechHub prints each product's spec badges and its rival; the Sport Outlet prints whether a court is built and the conversion penalty if not.

12.8 Staff sheet

One hire row per role, plus one read-only stats row per shop. A hire row shows a glyph on a tinted disc, the role name, a one-line status that changes when hired, and either a Hire button with its price or a greyed "HIRED".

The stats rows are shop-specific and are the honest scoreboard:

Shop	Stats row shows
Coffee	total tips earned, and walkouts
Burger Rush	trays out, sold, walkouts
Sport Outlet	conversion percentage, bought, rejected, walkouts
Boutique	conversion, sizes fetched, cubicles free
TechHub	conversion, advised, compared

12.9 Map sheet

The mall drawn as a ring: one box per shop, linked shop to shop, solid where you can ride and dashed where the far end is locked. A box carries the shop's glyph, its name and its state — owned, affordable, or its price. Below the ring, one row per shop shows what it earns unattended and a Travel button.

Tapping an owned box travels there. Tapping an affordable locked box buys it open. The middle of the ring keeps the one figure that belongs to the mall rather than to any shop: what the place earns while you are away.

12.10 Boost sheet

One purchase: 15 gems for x2 income for 60 seconds. While it runs, a banner sits under the HUD with the multiplier and a countdown.

12.11 Settings sheet

Reached from the gear in the HUD, not from the dock.

Group	Rows
Language	English, Simplified Chinese, Traditional Chinese, Bahasa Melayu
Save	Save now, Export, Import
Drive	Push, Pull, auto-backup toggle — or a line saying it is not set up
Danger	Reset, behind a confirm

12.12 Offline sheet

Shown on load when time has passed. It reports what the mall earned while away, capped at 2 hours at half rate.

12.13 In-world feedback

Most of the interface is not in the HUD at all. It is drawn on the floor.

Element	Meaning
Thought bubble	what this customer came for; two glyphs side by side for a two-item ticket
Green ring round the bubble	patience remaining; it only drains under that shop's own condition
Question mark over a head	they are wondering — stand with them and advise
Ruler over a head	they need a size fetched from the stockroom
Scales over a head	they are weighing the two rivals against their priority
Gold arrow on the floor	the tutorial's current target
Level pad	stand here to pour cash into the next level
Build plot	an unpaid fixture, with its price on it
Tinted floor block with a name	a department: VEGETABLES, GRILL, FITTING ROOMS
Rail label S 2 · M 0 · L 1 · XL 3	per-size stock, with a red pip for a size at zero
Red station load figure	the bottleneck in Burger Rush
Cash pile	walk over it, or wait 9 s and it banks itself

12.14 Notification language

Toasts last 1.9 seconds and never block. They are used for completions and confirmations — saved, reset, hired — and never for guidance. Guidance always goes to the objective line, because a toast that carries an instruction has disappeared by the time the player has walked anywhere.

All interface strings are keyed and translated. Product and shop names are also translated, falling back to the English original.

13 Art direction, brand and audio

There are no image assets in the game. Every product, fixture, animal and character is painted procedurally to the canvas at draw time. The only files under `assets/` are the app icon and wordmark, and those are generated too.

13.1 Identity

Name	Mini Shopping Mall — "MINI" on a gold slab, "SHOPPING MALL" in white with a navy outline
Descriptor	<i>Idle Tycoon</i> , letter-spaced under the wordmark
Mark	An isometric two-storey mall on a sky-blue tile, with a gold "M" billboard, a shopper and a stack of cash
Type	Baloo 2 at weight 800, falling back to Fredoka, Nunito, then system UI

13.2 The face shading rule

One rule governs every isometric solid in the game and in the brand: **three tints of one colour — top is the base, the right face one step darker, the left face two steps**. Any new prop must follow it or it will not sit in the same light as everything around it.

13.3 Palette

The primary colourway is "Sky".

Role	Hex	Used for
Sky light to dark	#7CD9FF to #2A6FD6	icon background gradient, theme colour
Mall white	#FFFFFF / #EFF4FB / #D7E2F1	base floor, three faces
Gold trim	#FFC53D / #F2AC28 / #DC961A	cornice, billboard, currency, primary buttons
Mall pink	#FF7BA6 / #F25F8F / #D9457A	upper floor, awnings, shoppers
Glass	#7FD4FF / #CDEEFF	shopfront windows
Money green	#5FE08D / #41C673 / #2CA85C	cash, income figures, the "earned" state
Ink	#16295C	text on gold, outlines, shadows at 13-20%

An alternate "Sunset" colourway swaps the sky for #FFC46B to #F4633F and the pink floor for teal #3FD4CB. It exists for seasonal skins and store-icon A/B tests. Money green is the one colour that may not be recoloured — it means income everywhere in the product.

Each shop also carries its own accent, used on the Map and the shop tag: Grocery mint #5FCBB6, Coffee brown #B07A4E, Burger Rush orange-red #E8552F, Sport Outlet violet #8B62FF, Boutique pink #FF7BA6, TechHub blue #4FB0FF.

13.4 Product painters

`src/art.js` holds one painter per product, about 1,500 lines of canvas drawing. The contract is fixed: a painter receives `(ctx, x, y, s, c)`, draws an item *standing on the point* `(x, y)`, grows upward by roughly `s` pixels, and tints with `c`.

Shapes are deliberately bold and few-sided, because the same painter draws the item at roughly 24 px on a shelf, in a customer's thought bubble, in the carry stack over the player's head, and as a chip in the Products sheet.

13.5 Logo pipeline

The brand kit is code, not artwork. `tools/build-logo.mjs` draws the isometric mall icon and the wordmark and writes every file in `assets/logo/`. Hand-editing an SVG there is a mistake — the next run overwrites it.

```
npm run logo:png
```

Output	Use
<code>icon.svg</code> , <code>icon-1024.png</code>	primary app icon
<code>icon-512/192/180.png</code>	Play Store, PWA, iOS home screen
<code>icon-simple.svg</code> , <code>favicon-32/64.png</code>	below 40 px, with props and window detail stripped
<code>icon-sunset.svg</code>	the alternate colourway
<code>wordmark.svg</code>	title screen, splash
<code>logo-lockup.svg</code>	horizontal icon plus wordmark
<code>preview.html</code>	every asset at real size, in a browser

The wordmark is white fill with a `#16295C` outline at 16 px and a gold slab offset 16 px down.

One shipping caveat: the generated SVGs reference the font by name and keep live text, so the tagline stays editable during development. **Before the wordmark goes anywhere public, convert the text to outlines**, or a machine without Baloo 2 renders a fallback.

13.6 Audio

Not built. There is no sound engine, no music and no effects.

When it is added, the cue list follows the feedback the game already gives visually: a pickup, a drop, a till ring, a cash pickup, a level-up, a build completion, a walkout, and one ambient bed per shop. The walkout cue matters most — it is the only failure the player can currently miss while facing the other way. Phase 4 proposes synthesising the cues in Web Audio rather than shipping sound files, which keeps the "no assets" rule and costs nothing.

14 Technical architecture

14.1 Stack

Vanilla HTML5 canvas and plain JavaScript. In development, scripts load as classic `<script>` tags into one `MSM` namespace — no bundler, no ES modules, no framework, no dependency at runtime. That choice is load-bearing: it is what lets a double-clicked `index.html` work from `file://`. Anything that requires a module graph breaks distribution as a folder.

The copy players get is **built**. `npm run build` joins the same 24 files, in the order `index.html` lists them, into one strict-mode function, minifies it with esbuild, and writes `dist/` with content-hashed file names and no source map. `dist/index.html` still opens by double-click.

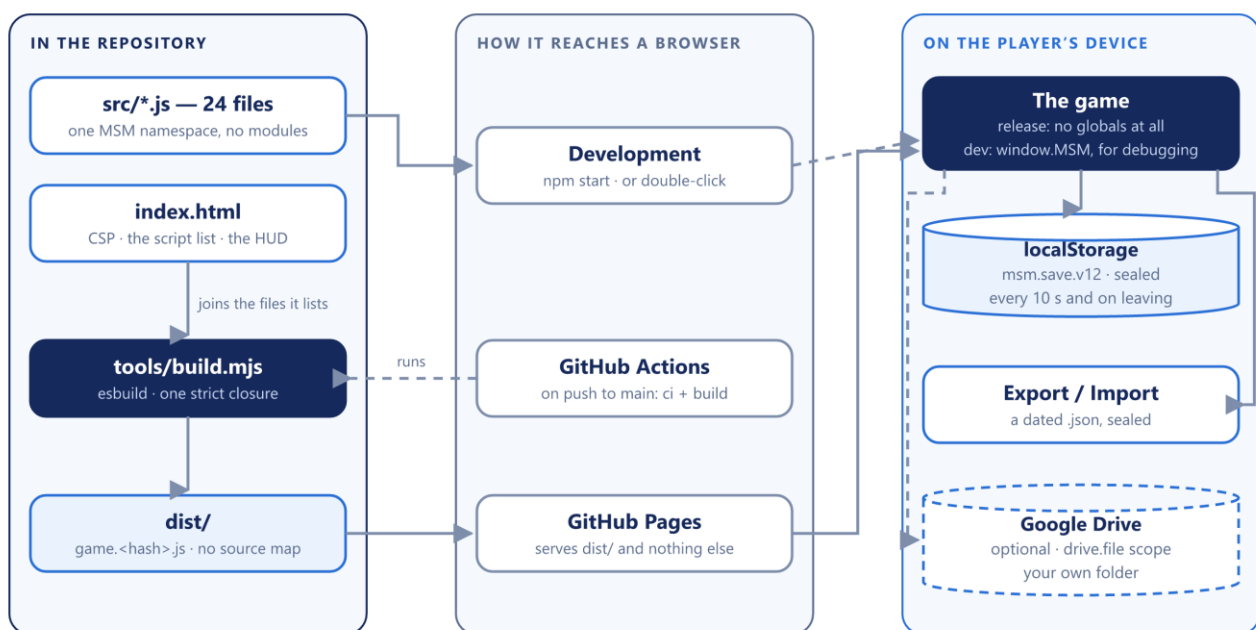


Figure 7 — System architecture — the same source, two ways into a browser

Layer	Choice	Why	Rejected
Language	Vanilla ES2020 JavaScript	No transpile step for development	TypeScript — a build for every edit
Rendering	One <code><canvas></code> , hand-written isometric renderer	Every product painted in code; no sprite sheets to load	A game engine — a runtime bigger than the game
UI	HTML HUD and bottom sheet over the canvas	Text that translates and scales; one template per sheet	Canvas-drawn UI — no accessibility, no text layout
Release build	esbuild, one closure	No globals, one file, hashed name	Webpack, Vite — a config for a concatenation
Storage	<code>localStorage</code> , one sealed key	Synchronous, so the loop never waits; a whole save is about 10 KB	IndexedDB — async for no gain at this size
Backup	Export <code>.json</code> + Google Drive <code>drive.file</code>	A file the player owns	A game server — cost, accounts and a privacy policy
Hosting	GitHub Pages via GitHub Actions	Free; deploys <code>dist/</code> on every push to <code>main</code>	Paid static hosts — nothing to buy
Local dev	<code>tools/dev-server.mjs</code> on port 8788	107 lines, no install	live-server — an npm tree for a static folder

14.2 File layout

File	Lines	Responsibility
<code>src/config.js</code>	1,363	balance, the store list, one floor plan per shop — tune here
<code>src/art.js</code>	1,528	one painter per product
<code>src/i18n.js</code>	1,575	four language packs and the lookup
<code>src/render.js</code>	1,376	the canvas scene and the joystick
<code>src/ui.js</code>	1,030	HUD, bottom sheets, toasts, the offered-button check
<code>src/entities.js</code>	797	player, stockers, customers, cash
<code>src/state.js</code>	660	state, economy maths, the seal, save/load, offline
<code>src/game.js</code>	601	loop, input, the till, unlocking, travel
<code>src/world.js</code>	297	what is solid, and how bodies move and slide
<code>src/tutorial.js</code>	241	the per-shop script and the live hint
<code>src/drive.js</code> + <code>drive-config.js</code>	244	the Google Drive copy
<code>src/iso.js</code>	147	isometric projection and the trailing camera
<code>src/backup.js</code>	113	export and import the save file
<code>src/cafe.js</code> + <code>cafe-render.js</code>	989	Coffee Shop
<code>src/food.js</code> + <code>food-render.js</code>	820	Burger Rush
<code>src/sports.js</code> + <code>sports-render.js</code>	668	Sport Outlet
<code>src/boutique.js</code> + <code>boutique-render.js</code>	833	Fashion Boutique
<code>src/tech.js</code> + <code>tech-render.js</code>	673	TechHub
<code>tools/build.mjs</code>	139	the protected release build
<code>tools/dev-server.mjs</code>	107	the development server
<code>tools/build-logo.mjs</code>	258	every file in <code>assets/logo/</code>

13,955 lines of game code in total, plus 282 lines of CSS and a 112-line page. The release build is one script of about 310 KB.

14.3 The shop module contract

A shop with its own game is declared by `mode` on its store entry, and implemented as a pair of files. Adding a shop means honouring this contract and nothing else.

The logic module publishes `MSM.<mode>` and must provide:

Method	Called by	Job
<code>active()</code>	the frame loop	is this shop the one being stood in
<code>reset()</code>	on load and on travel	clear in-flight state
<code>spawn()</code>	<code>entities.spawn</code>	make this shop's kind of customer
<code>stepCustomer(c, k, dt)</code>	<code>entities.updateCustomers</code>	advance one shopper's state machine
<code>update(dt)</code>	the frame loop	stations, plots, hired crew
<code>guide()</code>	tutorial	the live hint line for this floor

The render module publishes drawing helpers and, by convention, a `collect(items, ctx)` that contributes this floor's fixtures to the depth-sorted draw list.

Both files are loaded in `index.html` with the render module **after** the logic module. That ordering is a trap: both write into the same `MSM.<mode>` object, so a logic method named `collect` is silently clobbered by the renderer's. This was hit once in `food.js` and the logic method was renamed to `handOver`.

14.4 Floor plans

Every shop declares a plan in `MSM.CFG.PLANS` holding its world box, station boxes, level pads, shelves, lanes, sections, zones, till, queue slots, entrance, door, sign, bin and escalator. Shop-specific plans add their own arrays: `machines` and `tables` for the cafe, `areas` for courts and demo benches, `rooms` and `waits` for the boutique.

`MSM.CFG.usePlan(i)` swaps the active plan **in place** — it clears and reassigns the keys of the existing `MSM.CFG.PLAN` object rather than replacing it. Every module captured that reference at load time, so replacing the object would silently orphan all of them.

At load, each store's products are bound to its own plan: shelf boxes, lanes, browse points, output crates, level pads, machine index, court index, department section, and the derived `upgradeBase`.

14.5 Movement and collision

`world.js` maintains a walkability cache, rebuilt at the top of each frame. Bodies are circles of radius 0.22 that walk toward a target and slide along obstacles rather than stopping dead.

Only the player collides with fixtures. Customers and staff collide with each other — a separation pass runs after everyone has moved, so no two bodies end a frame in the same spot — but they pass through scenery and clip through fixtures on the diagonal. This is a known, accepted limitation, scheduled for Phase 4.

14.6 Rendering

One canvas, one depth-sorted pass per frame. Fixtures, products, bodies and floor markings are collected into a single list and drawn back to front by isometric depth. Product art, HUD chips and thought bubbles all call the same painters, so a new product needs exactly one function to appear everywhere.

14.7 Deployment

Pushing to `main` runs `.github/workflows/pages.yml: npm ci, npm run build`, and an upload of `dist/` — and only `dist/` — to GitHub Pages. The raw `src/` files are never served to players. Because the script and stylesheet

names carry a hash of their contents, a new release can never be served from a stale cache; there is no `?v=` to remember to bump.

15 Save data and persistence

The whole game state is one JSON object in `localStorage` under the key `msm.save.v12`, wrapped in a seal. It is written every 10 seconds of loop time, on `visibilitychange` and on `pagehide`. A quota error or a private-mode failure is swallowed silently — the game keeps running.

15.1 Shape

What is stored is `{ format: "msm.sealed", data, sig }`: the game as a JSON string, and a hash of that string. Inside `data`:

Field	Holds
<code>cash</code> , <code>gems</code> , <code>level</code>	the three headline numbers
<code>current</code>	which shop you are standing in
<code>boostLeft</code>	seconds of Rush Hour left, counted in game time
<code>lastSeen</code>	a timestamp, for offline earnings
<code>totalEarned</code> , <code>served</code>	lifetime tallies
<code>stores[]</code>	one entry per shop, by array index

Each store entry holds `owned`, `till`, `tillPaid`, `open`, `stockers`, `cashier`, its own `tut` progress, `sales`, `walkouts`, a `products[]` array, and exactly one mode block: `cafe`, `sports`, `boutique`, `tech` or `food`.

A product entry holds `built`, `buildPaid`, `level`, `shelf`, `out`, `feed`, `t` and `pay` — so a half-paid build plot and a half-paid level survive a reload.

In-flight work is deliberately **not** saved. Orders, tickets, trays, part-cooked food and customers on the floor live only in the sim, and a reload starts the floor empty. The one exception is the cafe's pickup counter: drinks already made stay where they were.

15.2 The save-index rule

This is the single most dangerous constraint in the project.

MSM.load() merges stores by array index. Inserting a shop in the middle of `CFG.STORES` silently loads the wrong shop's data into every shop after it. When Burger Rush was added at index 2, the Sport Outlet, the Boutique and TechHub all shifted, and the save key had to be bumped to `v12` so the old save was retired rather than mangled.

Any future insert must bump `SAVE_KEY` again. Appending at the end is safe; inserting is not.

That rule was acceptable while the game had no public players. It is not any more: the Pizza Restaurant belongs at index 3, and bumping the key would throw away every live save. Phase 4 therefore starts by storing each shop's `id` and merging by `id`, with a one-time read path for `v12` saves — see [Project plan and timeline \(§18\)](#).

A second safety net catches shape changes within a shop: if a stored product array is a different length from the current one, that shop starts fresh rather than mapping old numbers onto different lines. This is what saved the Coffee Shop when it was rebuilt from 4 shelf products into 16 ingredients and recipes.

15.3 Forward-compatible merging

Load never trusts the file. Every number must be finite and inside the range the game itself could have produced, or it falls back to its default: shelf counts to `SHELF_CAP`, output to `CRATE_CAP`, feed to `FEED_CAP`, levels to the cap, `current` to the store count, and a `lastSeen` in the future to now. Cash already paid into a pad is capped one dollar below its price, so stepping on it can never finish a build for free. New fields added since a save was written take their blank-state value.

One field is reconciled rather than read: the boutique keeps both a per-size breakdown and a rail total, which are two views of one thing, so the loader recomputes rather than trusting either.

Tutorial progress was once a single number for the whole save that in practice only described the mini mart. On load, an old save's number is applied to shop 0 only; every other shop starts its own walkthrough.

15.4 Export, import and Drive

Route	What it does
Export	writes a dated <code>.json</code> you can keep anywhere — sealed like the local save
Import	reads one back, after checking its seal
Drive push / pull	one file in a folder of your own, <code>drive.file</code> scope
Drive auto-backup	opt-in, debounced, silent

Export and Import both work from a double-clicked `index.html`. Drive does not — it needs the game served from a real origin, such as the live GitHub Pages site. The OAuth client is configured in `src/drive-config.js` in the same Google Cloud project as the two sibling apps.

The Drive layer is shared with MoneyFlow and FinSim. That sharing carries a known trap: one Cloud project and one client ID across apps, with a per-app sub-folder, and changing an app's identifier orphans the save already in Drive.

Restoring always replaces, never merges. It confirms first, then reloads.

16 Security and anti-cheat

A single-player game with no server has no password database to leak and no other player to harm. Its threat model is short but real: a player who pastes `MSM.state.cash = 1e12` into the console, or edits a number in their save, has made the whole progression ladder meaningless — and a page that runs whatever code reaches it is also a page an injected script could hijack. The hardening of 23 September 2026 closed both, on the principle that no single mistake should reopen either.

16.1 Threat model

Threat	How it would arrive	Status
Setting cash from the console	<code>MSM.state.cash = 1e12</code>	Closed in the release build — no globals exist
Reaching inside through a built-in	A getter or <code>toJSON</code> planted on <code>Object.prototype</code> ; a replaced <code>forEach</code> ; a <code>requestAnimationFrame</code> that lies about the time	Closed — the game keeps its own copies and freezes the shared prototypes before any console code runs
Reading the code from the live site	View source, the Sources panel	Only one minified file is served; no <code>src/</code> , no source map
Editing the save	Application → Local Storage, or an exported <code>.json</code>	Refused — a broken seal is rejected on load and on import
Impossible numbers in a save	<code>Infinity</code> , <code>NaN</code> , negatives, a level of 10,000	Clamped to what the game could produce
Rewriting a button	Changing <code>data-i</code> or re-enabling a disabled button in Elements	Ignored — a click runs only if the sheet offered it; every action re-checks its rules
Winding the clock back	Replaying a Rush Hour	Closed — the boost counts down in game time
Script injection	A <code><script></code> or <code>onerror=</code> reaching the page	Refused by the Content-Security-Policy
A refused save overwriting the cloud	A fresh game auto-pushing over the good Drive copy	Closed — auto-push stays off for the session
Someone with the DevTools debugger	A breakpoint, and a variable edited by hand	Out of scope — only a game server could stop it, and it affects only their own copy

16.2 Controls in place

- **One closure, no globals.** `tools/build.mjs` joins the 24 files inside a single strict-mode function with `MSM` as a local variable, so nothing the game owns is reachable by name from the console.
- **Trusted built-ins.** Before any console code can run, the build takes its own frozen copies of `Math`, `JSON`, `Object`, `Date`, `performance` and `requestAnimationFrame`, and freezes `Object.prototype`, `Array.prototype`, `Function.prototype` and the array iterators. Four names — `constructor`, `toString`, `valueOf`, `toLocaleString` — stay assignable on a library's own objects, so Google sign-in keeps working. A plain freeze was chosen over getter/setter pairs after the latter measured 12x slower on the game's array code.
- **A namespace with no prototype.** `MSM` is created with `Object.create(null)`, so reading a property it does not have cannot land in someone else's getter.

- **The seal.** Every save and export is `{ format, data, sig }` with a 64-bit hash of the data and a pepper. An edited file fails the check and is refused — never destroyed: it is set aside under `msm.save.v12.rejected`, so a false refusal can still be recovered.
- **A legacy window.** Saves and backup files written before the seal are still read until **1 November 2026**, so nobody's progress was lost the day it shipped. After that date an unsealed save looks exactly like a hand-written one and is refused.
- **Clamped on the way in.** Every loaded number is checked — see [Forward-compatible merging \(§15.3\)](#).
- **Only offered buttons act.** Each sheet records the action and index of every button it rendered, read from its own HTML string rather than the page. A click on anything else does nothing, a mutation observer puts rewritten attributes back on the next frame, and every action in `game.js` then re-checks its index, ownership, cost and preconditions anyway.
- **Content-Security-Policy.** `index.html` allows scripts only from this site and Google's sign-in client, network calls only to Google's APIs, and sets `object-src 'none'`, `base-uri 'none'` and `form-action 'none'`.
- **A console warning.** The release build prints a "Stop!" warning telling players that code pasted into the console will not work — aimed at the scam that works on people, not on code.
- **A harder dev server.** `tools/dev-server.mjs` answers malformed URLs with 400, will not serve `.git`, `node_modules` or dotfiles, and sends `nosniff`, `DENY` framing and a strict referrer policy.

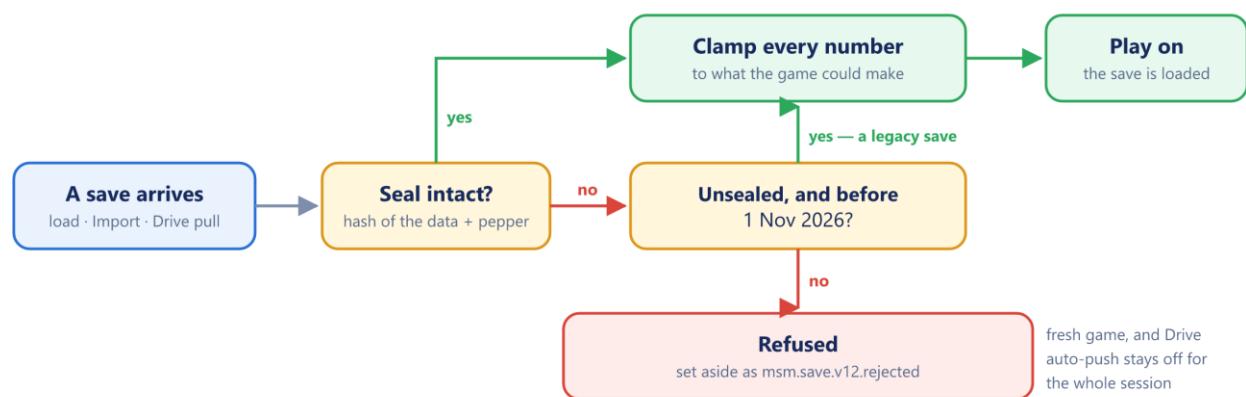


Figure 8 — How a save is checked — on load, on Import and on a Drive pull

16.3 Privacy by construction

No account, no analytics, no telemetry and no advertising SDK. The only network calls the page is permitted to make are Google's sign-in and Drive APIs, and only after the player chooses to connect Drive. The save lives in the player's own browser and nowhere else unless they export it.

16.4 What is not protected, and why

- **The debugger.** Anyone can pause the game on a breakpoint and edit a variable by hand. No web page can prevent that; only a server-authoritative game could, and it would only ever change the cheater's own copy.
- **A reader of the source.** The repository is public and the seal's pepper ships inside the game, so someone who reads the code can forge a seal. The seal stops the ten-second cheat, not a determined programmer. Hiding the pepper means making the repository private.
- **Unsealed saves until 1 November 2026.** Accepted on purpose, and closed automatically on that date.
- **The development build.** `npm start` serves `src/` with `window.MSM` in place, because that is what makes debugging possible. It is never deployed.

17 Testing and quality assurance

A build is judged by playing it, not by reading its numbers. A game's failure mode is rarely a crash — it is a floor that plays badly, a hire that quietly earns less, or a save that loads into the wrong shop — so the checks below are built around play.

Layer	How	Catches
Playtest	Drive the character through each shop by hand, from a fresh save and from a seeded one	Cramped floors, accidental triggers, missing guidance, a shop that is no fun
Measured runs	Fixed-length runs with and without each hire, read from the Staff sheet	A hire that lowers revenue; a bottleneck that does not move
Invariants	Console checks in the development build	Size breakdowns that disagree with the rail; negative stock
Save round trip	Export, clear site data, Import; and a reload mid-build	Lost progress, half-paid pads, the index rule
Security	Console attacks, edited saves and rewritten buttons against the release build	Anything that turns a typed number into cash
Headless capture	A script driving headless Edge over the DevTools protocol	The page failing to start; every shop rendering. It produced the screenshots in this document

17.1 Measured results on record

Measurement	Result
Burger Rush, fryer levelled, 80 s	fryer load 3.7 → 0.4, grill 1.2 → 3.0, trays out 39 → 44
Burger Rush, nobody hired	15 orders taken, 0 trays out, 9 walkouts
Boutique, 2 vs 5 fitting rooms	41% vs 66% conversion
TechHub, cold / benches / benches + advisor	28% / 62% / 70% conversion
TechHub first cut, advised vs demo-and-compare	54% vs 64% — the bug that produced the rule in The known balance lesson (§10.7)

17.2 The invariant worth testing

Per-size rack counts must sum to the rail total. A browser-console loop over `MSM.econ.bstate().racks` in the development build catches that whole class of bug in seconds, and the loader already re-normalises rather than trusting the save.

17.3 Browser matrix

Browser	Status
Chrome / Edge, desktop	Primary target; development and this document's screenshots
Chrome, Android	Target platform; to be verified on devices in Phase 4
Safari, iOS and macOS	To verify in Phase 4 — Safari may clear script-written storage after seven days without a visit
Firefox	Expected to work; to verify in Phase 4
Private / incognito windows	Plays; the save is lost when the window closes

17.4 Per-floor acceptance

- ☐ Every gap you can see, you can walk through — at least 1.0 wide against a 0.22 body
- ☐ Walking a lane never triggers a station you did not mean to touch — lanes sit clear of REACH 0.8
- ☐ Every station that eats something stands within a few units of what it eats
- ☐ The objective line is never blank while there is something useful to do
- ☐ No locked row anywhere says only "unlocks later" — it names the line ahead of it and a cost
- ☐ Every hire is visible on the floor doing its job
- ☐ The guidance script ends at the first sale and never runs again for that shop

17.5 Per-shop acceptance

- ☐ The shop's signature mechanic is the thing that decides whether you earn, not a price modifier
- ☐ With nobody hired, the shop is playable by hand and clearly worth playing
- ☐ With everyone hired, the shop earns unattended, and hiring strictly beats not hiring
- ☐ The Staff sheet reports a number that tells you whether the floor is coping
- ☐ Its failure state is visible on the floor, not only in a tally

17.6 Economy acceptance

- ☐ The next thing you can afford is always legible from the dock badges
- ☐ No level, build or hire leaves the player with nothing to do next
- ☐ Offline earnings are capped at 2 hours and half rate, and are reported on return
- ☐ Money notation stays inside its pill at TechHub scale

17.7 Release checklist

- ☐ Every acceptance list above re-run on any shop that changed
- ☐ `npm run build` succeeds and `dist/index.html` plays from a double-click
- ☐ In the built game, `MSM` is undefined in the console and a pasted cash edit does nothing
- ☐ An edited save is refused and set aside as `.rejected`; an untouched one loads
- ☐ Export → clear site data → Import restores the game
- ☐ `SAVE_KEY`, or the id-based merge, confirmed against the final store order
- ☐ No console errors and no Content-Security-Policy violations
- ☐ Pushed to `main`, the Pages workflow green, and the live site checked with a hard reload

18 Project plan and timeline

Three phases are complete. Phase 4 is what this proposal asks approval for: ten weeks from 5 October to 13 December 2026, closing objectives O1, O13 and O14 and releasing v1.0.

18.1 Phases 1–3 — delivered

Phase	Dates	Delivered
1 · Foundation	27–28 Aug 2026	Grocery Store, the isometric engine, save layer, Drive copy and brand kit on day one; joystick reworked; camera containment; worst product art redrawn; four languages; Coffee Shop
2 · The roster	1–4 Sep 2026	Cafe chef and oven; Sport Outlet, Fashion Boutique and TechHub on 1 Sep; Burger Rush as shop 3 on 2 Sep, with the save key bumped to v12; per-shop guidance; playtest pass on space and guidance; level cap 100; character and interface polish
3 · Proposal and hardening	22–23 Sep 2026	Revision 1 of this proposal; the protected release build, sealed saves, clamped loads, offered-button checks, game-time boost, Content-Security-Policy, a hardened dev server and the GitHub Actions deploy

Six shops, four languages, the brand kit, the save layer and the Drive copy all landed inside nine days. Twenty-nine commits in all.

18.2 Phase 4 — proposed

Week	Dates	Work	Deliverable
1	5–11 Oct	Save by shop id — each store entry carries its <code>id</code> , load merges by id, and a one-time path reads <code>v12</code> saves into the new order	Pizza can be inserted at index 3 without retiring a single save
1–3	5–25 Oct	Pizza Restaurant — floor plan budgeted for walking room first; the sequential station chain; oven timing with a burnt state; difficulty tiers; its hires; its guidance script	Shop 4 playable; objective O1 met
4	26 Oct – 1 Nov	Seal cutoff — confirm legacy saves still load on 31 Oct and are refused on 1 Nov; prompt one Export for players who have never made one	The legacy window closes cleanly
4–5	26 Oct – 8 Nov	Sound — a Web Audio engine and the eight cues, synthesised rather than recorded; a mute switch in Settings	Objective O14 met
6	9–15 Nov	Collision for staff and shoppers — steer around fixtures instead of clipping through them	No visible clipping on any floor
7–8	16–29 Nov	Late-game balance — seed a save at each unlock tier, play each through, tune <code>config.js</code>	Objective O13 met; the \$95M unlock actually reached
9	30 Nov – 6 Dec	Release prep — PWA manifest and install icons, wordmark converted to outlines, the browser matrix verified on real devices	Installable from the browser
10	7–13 Dec	Release, documentation refresh, proposal revision 3	v1.0

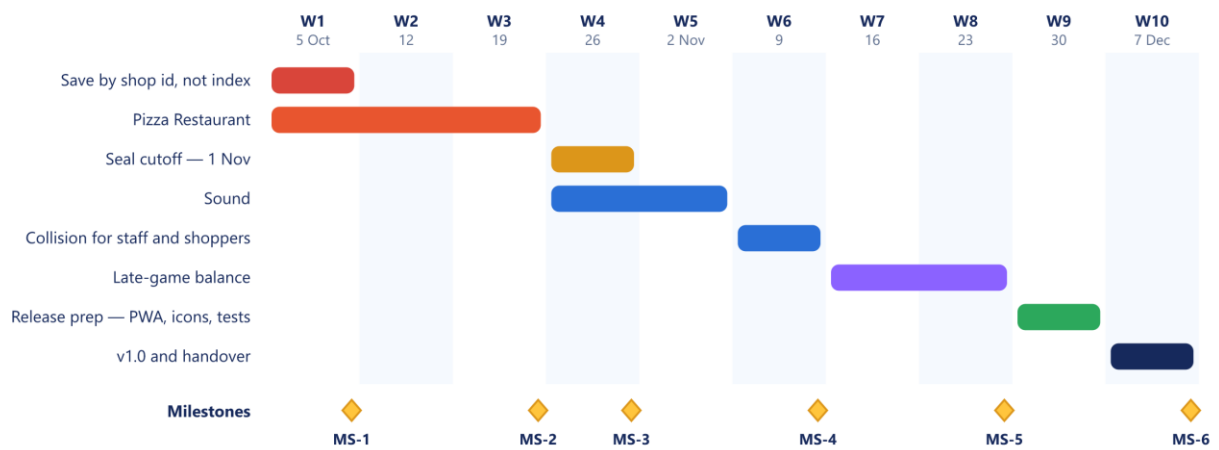


Figure 9 — Phase 4 — ten weeks, six milestones

18.3 Milestones

ID	Milestone	Due	Gate
MS-1	Id-based save merge live	2026-10-11	A v12 save loads into the new order unchanged
MS-2	Pizza Restaurant playable	2026-10-25	Per-floor and per-shop acceptance lists pass
MS-3	Seal cutoff passed	2026-11-01	Legacy saves refused; no false refusals reported
MS-4	Sound and collision in	2026-11-15	Every cue fires; no clipping on any floor
MS-5	Late game balanced	2026-11-29	Every unlock tier played through
MS-6	v1.0 released	2026-12-13	Live, documented, tagged

MS-1 is a hard gate. Building the pizza floor before the save merges by id would force exactly the choice the index rule describes — retire every save, or mangle them — and the game now has players it would do that to.

19 Risks and mitigations

ID	Risk	Likelihood	Impact	Mitigation	Residual
R1	Inserting the pizza shop shifts every later shop's save	Certain if unmitigated	Critical	MS-1: merge by shop id before any insert; v12 read path	Low
R2	The save has one copy, in one browser — clearing site data loses the game	Medium over a year	High	Export, Drive push and opt-in auto-backup; a one-time Export prompt in week 4	Medium — all need the player to act once
R3	Safari clears script-written storage after seven days without a visit	Medium	High	Drive copy; verify on iOS in week 9; document it	Medium — not fixable client-side
R4	The seal refuses a genuine save	Low	High	The save is set aside, not deleted; Drive auto-push stays off; recovery is possible by hand	Low
R5	A cheater forges a seal from the public source	Low	Low	Accepted — it affects only their own copy; a private repository is the only real fix	Accepted
R6	Late-game balance is unverified past the early shops	High	Medium	MS-5: seed a save at each tier and play it	Low after MS-5
R7	No automated test suite; a regression is found only by playing	Medium	Medium	Acceptance lists; Staff-sheet figures as smoke tests; the headless capture script as a start-up check	Medium
R8	Staff and customers clip through fixtures	Certain today	Low	Week 6	Low
R9	A render module's helper silently replaces a logic method of the same name	Low	Medium	Naming rule in the module contract	Low
R10	Single developer, shared with the sibling apps' own next phases	Medium	High	This document as the handover; the module contract; config.js as the one place to tune	Medium
R11	The wordmark ships as live text and renders in a fallback font	Medium	Low	Convert to outlines in week 9	Low
R12	A change to the shared Drive layer or Cloud project breaks all three apps	Low	Medium	One client ID per app, one sub-folder per app; test Drive in all three after any change	Low

19.1 The three that deserve a decision, not just a mitigation

R1 is the reason for MS-1. The index rule was safe while nobody else played. Now it is a choice between retiring every save and mangling them, and merging by id removes the choice.

R2 and R3 are the same risk. Both end with a browser that no longer holds the game. The only real answer is a second copy that is not the browser, and today that copy exists only if the player makes it. Phase 4 asks for it once, rather than waiting for the player to discover Settings.

R10 is a scheduling risk, not a technical one. The same developer owns MoneyFlow and FinSim. Phase 4 is sized at ten hours a week so that it can run alongside them; if another project's phase takes the same weeks, the plan slips as a whole rather than being squeezed.

20 Resources and budget

Mini Shopping Mall costs RM 0 a month to run today, and Phase 4 keeps it there. The only real resource is developer time.

20.1 People

Role	Who	Commitment
Developer, designer, artist and tester	One person	~10 hours a week for 10 weeks — 100 hours
Playtester and product owner	The same person, plus friends and family on the live build	Continuous

A second developer is not proposed. This document, the module contract and `config.js` are what a handover would run on.

20.2 Tools and infrastructure

Item	Cost	Note
GitHub repository, Actions and Pages	RM 0	Free for public repositories
Storage	RM 0	The player's own browser
Google Cloud project + Drive API	RM 0	Shared with the sibling apps; free at this volume
Node.js, esbuild, resvg	RM 0	Build-time only; none ships to players
Fonts	RM 0	Baloo 2 is open-licensed
Sound	RM 0	Synthesised in Web Audio — no files to buy
Art	RM 0	Painted in code
Current total	RM 0 / month	

20.3 Optional and conditional costs

Item	Cost	When it applies
Custom domain	~RM 60 / year	Only if the <code>github.io</code> address is not wanted
Google Play developer registration	US\$25 once (≈ RM 106)	Only if the game is wrapped and listed on Android
Apple Developer Program	US\$99 / year (≈ RM 421)	Only if the game is wrapped and listed on iOS

Ringgit figures are approximate, converted at about RM 4.25 to the US dollar; they are for sizing, not for budgeting to the sen.

20.4 Three-year total cost of ownership

Scenario	Year 1	Years 2–3	3-year total
Web only (as proposed)	RM 0	RM 0	RM 0
Web + custom domain	RM 60	RM 120	RM 180
Web + Android listing	~RM 106	RM 0	~RM 106
Web + Android + iOS listings	~RM 527	~RM 842	~RM 1,369

The recommendation is to stay web-only through v1.0 and decide on store listings with the monetisation question in [Future roadmap \(\\$21\)](#), because a listing without a plan for revenue is a cost with nothing to set against it.

20.5 What the budget does not buy

No paid art, no paid audio, no analytics, no crash-reporting service, no advertising network and no server. Everything in the stack is free at this scale because the game is deliberately small enough to stay there.

21 Future roadmap

After v1.0 the work splits three ways: what each shop's own backlog holds, what cuts across the mall, and what has been deliberately left out.

21.1 Next: shop 4, the Pizza Restaurant

The only gap in the roster, and the core of Phase 4. It is specified and deliberately contrasted against Burger Rush: fast food cooks three things **in parallel**, pizza walks one thing through stations **in sequence**.

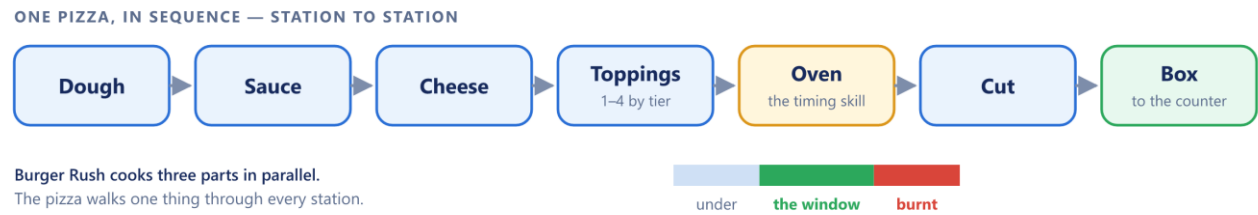


Figure 10 — The Pizza Restaurant — one pizza through every station, with the oven as the skill

The skill is oven timing — undercooked and burnt are both failures — and each pizza carries a difficulty tier from one star (Cheese) to four (Deluxe), which sets how many topping steps it needs.

Definition of done for shop 4:

- ☐ A floor plan budgeted for walking room before the maths, per the playtest criteria
- ☐ A sequential station chain where a pizza physically moves between stations
- ☐ Oven timing with a visible window, and a burnt state that loses the sale
- ☐ Difficulty tiers that change the number of steps, not just the price
- ☐ Its own guidance script ending at the first sale, plus a live hint line
- ☐ Saves merged by shop id first, so inserting it at index 3 retires nothing
- ☐ A hire that appears on the floor as a body

21.2 Deferred by shop

Each built shop has a named backlog. None of it blocks play.

Shop	Deferred
Coffee	day cycle, end-of-day summary, missions; further food and equipment tiers
Sport Outlet	quality tiers and brands, player-set prices, seasonal promotions, store reputation and VIP athletes, the further four sports the zones are shaped for, a trial animation instead of a progress bar
Boutique	colour as a second axis, outfit style matching, trends and seasons, promotions and bundles, VIP shoppers, assistant levels
TechHub	delivery for big boxes, a service desk with warranty and repairs, trade-ins, installation, storage and colour variants, launch events, supplier tiers

21.3 Cross-cutting candidates

Item	Value	Cost
Prestige	A reset-for-multiplier layer that gives the late game a reason to start again	Medium — a new balance axis
Per-store decoration	Something to spend on that is not throughput	Medium — must not break walking room
Tutorial past the first sale	A guided second hour	Low — the hint line already carries most of it
A second till per shop	Relief for the busiest floors	Medium — queue logic assumes one
Mobile store listings	Reach beyond the browser	See Resources and budget (\$20)

21.4 Deliberately not built, and why

- **Energy timers and adverts.** They make money by interrupting the loop, and the loop is the product.
- **A leaderboard.** An honest one needs a server-authoritative game; a dishonest one is worse than none.
- **Free placement of fixtures.** Every floor is authored so that walking room and feed distances are guaranteed; free placement gives both up.
- **Tap-to-collect.** Built first, and discarded.

21.5 Open questions

- **What unlock price does the Pizza Restaurant take?** It sits between Burger Rush (\$45K) and the Sport Outlet (\$180K) in the roster, which leaves a narrow band, or the shops after it shift.
- **Does the roster stop at seven?** The Sport Outlet's zones are shaped to take four more sports, which is a shop's worth of content without a new shop.
- **Is there a monetisation model?** Gems are earned only from mall levels today. Nothing in the build sells them, and nothing should until the question is answered on purpose.
- **What is the release platform?** The PWA and iOS icon sizes are generated, but there is no manifest, no service worker and no store listing yet.

21.6 The sibling apps

App	Domain	Relationship
MoneyFlow	Personal finance	Shares the Google Drive save layer and Cloud project
FinSim	Financial calculators	Shares the Drive layer; nothing else

What travels between them is the Drive layer and the discipline of a static site with no server. What does not travel is anything visual: Mini Shopping Mall is a game with its own brand.

22 Conclusion and approval

Mini Shopping Mall already works. Six shops are live and each plays as a different game on one set of controls, the save survives reloads and backups, the running cost is RM 0 a month, and since 23 September the release build refuses the cheats a single-player web game realistically faces. What this proposal asks for is not a build from nothing — it is ten weeks to finish the roster, give the game a voice, and play the late game for real before calling it v1.0.

The case rests on three things:

1. **The hard part is done.** The engine, the six shop modules, the module contract, the save layer and the security model exist and are documented here in full.
2. **The remaining scope is small and known.** One shop, specified. One sound engine, with its cue list already written. One collision pass and one balance pass. MS-1 exists so the only genuinely dangerous step — the save order — is solved before anything is built on it.
3. **The downside is bounded.** If Phase 4 stops early, every milestone reached still ships: the id-based save, the seventh shop, sound and balance are each independent improvements to a game that already plays — at RM 0.

22.1 The decision requested

Approve	Phase 4 as scoped — 10 weeks, ~100 hours, one developer
Budget	RM 0 committed. Store listings, if wanted, are a separate decision after v1.0
Decide by	2026-09-30
Start	2026-10-05

If Phase 4 is not approved, the recommendation is to do MS-1 anyway: merge saves by shop id. It is a week's work, and it is what keeps every future change to the roster from costing players their progress.

22.2 Sign-off

Role	Name	Decision	Date
Product owner			
Developer			
Reviewer			

22.3 Sources

Every figure in this proposal is taken from the Mini Shopping Mall repository as it stands on 23 September 2026 (commit [d4c0d1f](#)) — the source files in [src/](#) and [tools/](#), [index.html](#), [README.md](#), [docs/DRIVE.md](#),

`assets/logo/BRAND.md`, and 29 commits of git history from 27 August to 23 September 2026. The measured runs are the ones recorded during development. The screenshots were captured from the running game on 23 September 2026. The genre comparisons in [Background and problem statement \(§2\)](#) and the platform fees in [Resources and budget \(§20\)](#) are stated from general knowledge and should be checked against current terms before the document is shown outside the project.

23 Glossary

Term	Meaning
Arcade-idle	A casual genre where you move a character to do the work by hand, then hire staff to automate it
Idle clicker	A genre where progress comes from tapping menus and waiting, with no character in the world
Dimetric / isometric	The 2:1 angled grid the floors are drawn on
Tick	One pass of the frame loop
REACH	0.8 units — how close you must stand to use a station
Build plot	A fixture not yet paid for; stand on it and cash drains in
Level pad	The pad beside a station; stand on it to pay for the next level
Line	One product a shop sells, with its own station, level and price
Stocker	The generic hire who hauls feed and restocks shelves
Patience	The ring round a customer's bubble; at zero they walk out
Grace	Seconds before patience starts to drain
Conversion	Of everyone who walked in, the share who bought
Bottleneck	The slowest station in Burger Rush, which sets the whole shop's pace
Mall level	A level earned from the sum of every product level you own
Gems	3 per mall level; spent only on a Rush Hour
Rush Hour	x2 income for 60 seconds, for 15 gems
Offline earnings	What staffed shops earn while the game is closed — up to 2 hours at half rate
SAVE_KEY	The <code>localStorage</code> key the save lives under, currently <code>msm.save.v12</code>
Seal	The hash stored beside the save; an edited save breaks it and is refused
Pepper	The fixed secret mixed into the seal's hash; it ships inside the game
Closure	A function whose local variables nothing outside it can reach; the release build is one
CSP	Content-Security-Policy — a rule in the page that tells the browser which scripts it may run
drive.file	The narrowest Google Drive permission: only files the app itself created
Content hash	A short fingerprint of a file's contents in its name, so a changed file always gets a new name
PWA	Progressive Web App — a web page that can be installed like an app